

Architecture — Distributed Document Search Service

A multi-tenant search service prototype targeting 10M+ documents, p95 < 500ms, and 1000+ concurrent searches/sec, with horizontal scalability and tenant isolation as first-class constraints. Postgres is the source of truth; OpenSearch is the search projection; Redis is cache, rate limiter, and queue.

High-level system

The API tier is stateless FastAPI behind an L7 load balancer. State lives in three backends — Postgres, Redis, OpenSearch — and a separate worker pool drains the index queue.

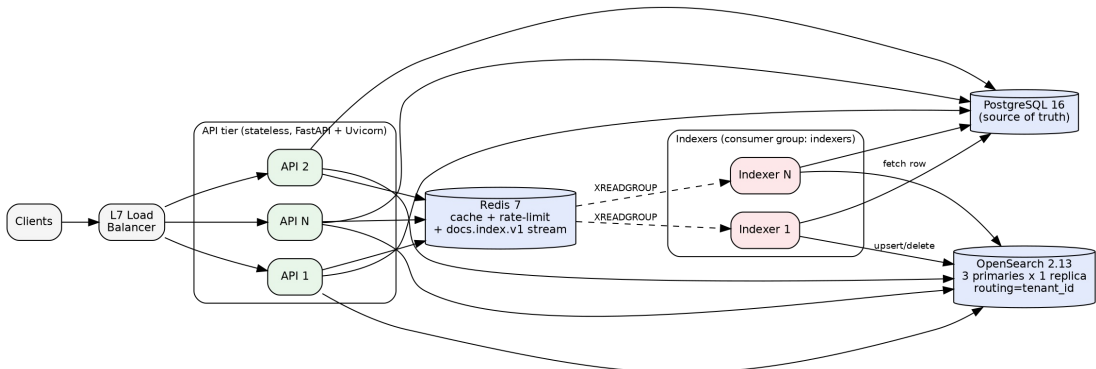


Figure 1 — High-level system

Every component scales horizontally: API instances are stateless containers, OpenSearch shards across nodes, Redis can shard by key prefix, and indexers consume in parallel via a Redis Streams consumer group.

Data flow

Indexing (write path)

The write path commits to Postgres synchronously and queues the search index update asynchronously. The 202 returns only after a durable PG commit, so a client that immediately calls `GET /documents/{id}` reads its own write.

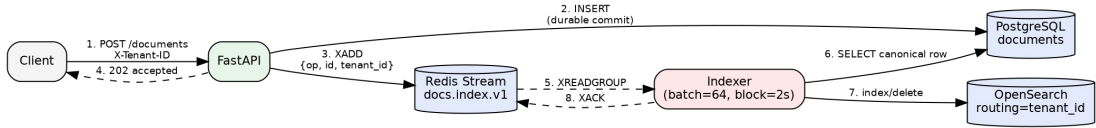


Figure 2 — Indexing data flow

The indexer batches `XREADGROUP` calls (size 64, block 2s), fetches the canonical row from Postgres rather than trusting the message body, and upserts into OpenSearch with `routing=tenant_id`. `XACK` only fires after a successful OpenSearch response. Delete follows the same path with `op: delete` and a soft-delete (`UPDATE documents SET deleted_at = now()`).

Search (read path)

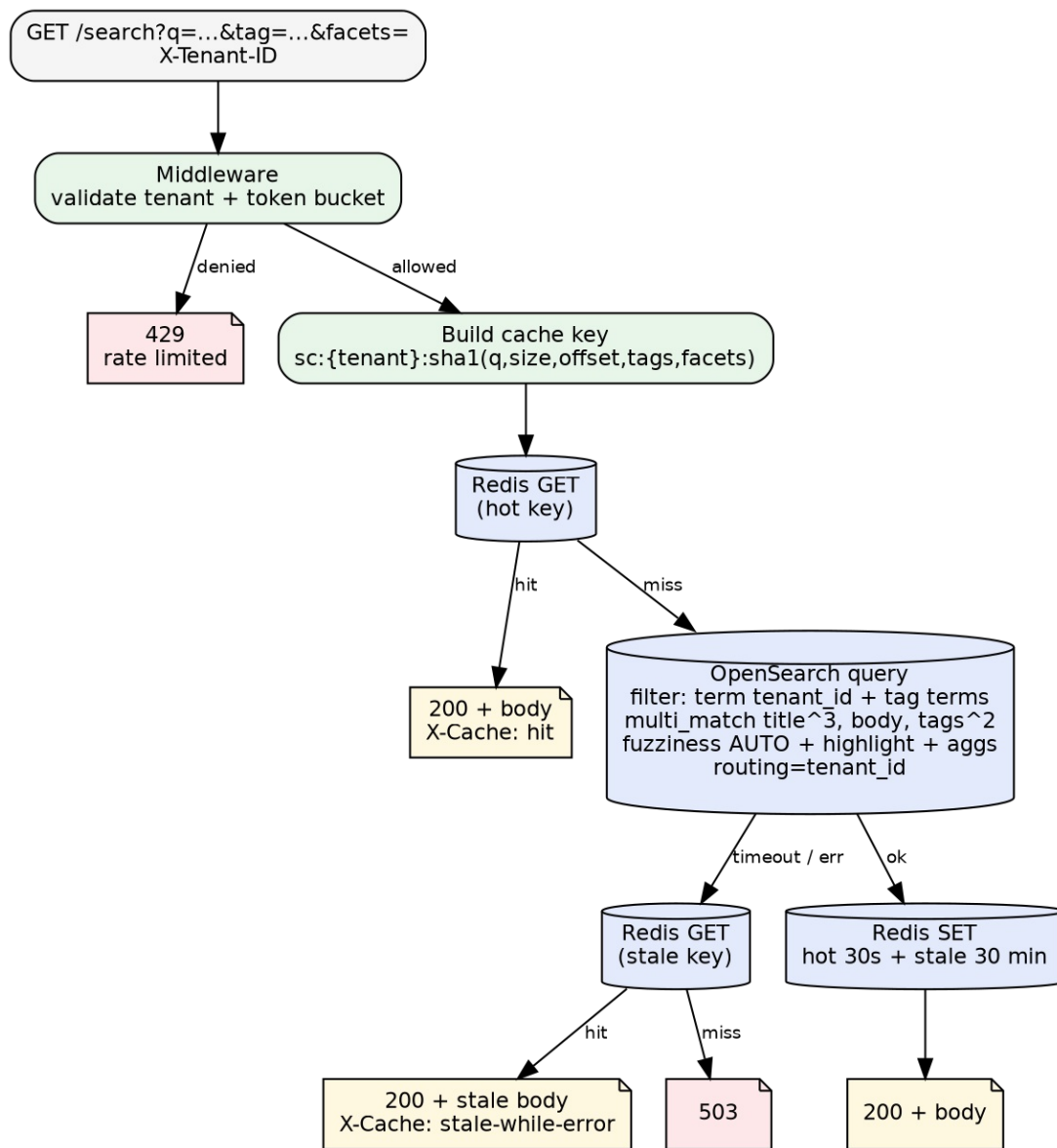


Figure 3 — Search data flow

Tenant identity is sourced from `request.state.tenant_id` — set by middleware from the validated header — never from the query. The OpenSearch query is a `multi_match` on `title^3`, `body`, `tags^2` with `fuzziness: AUTO` and highlighting, wrapped in a bool filter that always includes `term: {tenant_id}` and any `tag` terms. `routing=tenant_id` ensures the query hits one primary shard.

Storage strategy

Store	Role	Why
Postgres 16	Source of truth	ACID writes, JSONB metadata, partial index on <code>deleted_at</code> IS NULL for active rows. Search engines are opinion-y projections — never the only copy.
OpenSearch 2.13	Search projection	Inverted index, BM25, highlighting, fuzziness, native shard routing. Postgres FTS doesn't scale past tens of millions on a node and has no per-tenant routing. OpenSearch over Elasticsearch because the ES license change pushed the open ecosystem here.
Redis 7	Cache + rate limit + queue	Sub-millisecond cache, atomic Lua for token buckets, Streams for at-least-once consumer groups without standing up Kafka.

Postgres holds `tenants(tenant_id, rate_limit_rps)` and `documents(id, tenant_id, title, body, tags TEXT[], metadata JSONB, created_at, updated_at, deleted_at)`. OpenSearch runs a single shared index `documents` with 3 primaries × 1 replica; mapping has `tenant_id` as keyword, `title` as text + raw keyword, `body` as text, `tags` as keyword, `metadata` as object.

If OpenSearch corrupts, we rebuild from Postgres by replaying inserts onto the stream.

API design

```

POST /documents          202 {id, status: "accepted"}
GET /documents/{id}      200 {id, tenant_id, title, body, tags, metadata, timestamps}
DELETE /documents/{id}   202 {id, status: "accepted"}
GET /search?q=&size=&offset=&tag=&facets=  200 {took_ms, total, hits[], cache, facets?}
GET /health              200 {status, dependencies{postgres, redis, opensearch}}
```

All non-public endpoints require `X-Tenant-ID`. Errors return `{error, detail}`. Rate-limited responses carry `X-RateLimit-Limit` and `X-`

`RateLimit-Remaining`. `/search` accepts repeatable `tag=` for AND filtering and `facets=true` for tag bucket counts via `aggs.tags.terms`. Writes return 202 because the search index update is async — the durable commit has happened, but the document may not be searchable yet.

Consistency model

Writes are strongly consistent in Postgres — the 202 follows a durable commit. Search is eventually consistent: P50 visibility lag is the indexer batch drain plus OpenSearch's refresh interval (default 1s). Clients that need read-your-write semantics for a specific document call `GET /documents/{id}`, which reads Postgres directly; only full-text search is eventual.

The cache uses TTL-based expiry with no active bust on writes. Bounded staleness is 30 seconds. We chose this over write-path invalidation because invalidation across cache keys derived from `q + tags + facets_flag` is unbounded, and 30 seconds is acceptable for search UIs.

Caching strategy

Three layers, each tenant-prefixed so cross-tenant cache hits are structurally impossible.

Layer	Content	TTL	Key
L1 hot	<code>/search</code> response body	30s	<code>sc:{tenant_id}:sha1(tenant\ q\ size\ offset\ sorted_tags\ facets_flag)</code>
L1 stale	same body, graceful-degrade copy	30 min	<code>...:stale</code> suffix on the same key
L2	OpenSearch request cache (filters, aggs)	OS default	OS-managed
L3	OS shard page cache	implicit	OS workload

The stale layer exists for the failure mode below — when OpenSearch is wedged, the search route catches the timeout and returns the stale body with `X-Cache: stale-while-error`. If no stale entry exists, it returns 503.

Message queue

`docs.index.v1` is a Redis Stream with consumer group `indexers`, `MAXLEN ≈ 100k` approximate trim. Consumers scale horizontally; each `XREADGROUP` returns up to 64 entries.

Failure handling has three layers. The indexer's main loop runs `XAUTOCLAIM` against entries idle > 30s and retries them. Each retry consults `XPENDING` for `times_delivered`. After `indexer_max_deliveries` (default 5) the entry is `XADD`ed to `docs.index.dlq` and `XACK`ed off the main stream — a poison message can't block the partition. The DLQ (`MAXLEN ~10k`) is the alerting surface.

We picked Redis Streams over Kafka because we already run Redis for cache and rate-limiting; one binary, three jobs. The migration trigger is sustained ~50k msg/s, at which point Streams' single-node ceiling becomes the bottleneck. In production, cache Redis and stream Redis run as separate clusters so cache eviction can't drop stream state.

Multi-tenancy and data isolation

Tenancy is enforced in five places, with a single source of truth — `request.state.tenant_id` set by middleware from the validated `X-Tenant-ID` header:

- 1. **Authentication** — middleware validates the header against the `tenants` table on every request. The prototype trusts the header; production replaces this with a signed JWT carrying the tenant claim.
- 2. **Query injection** — every OpenSearch query gets a mandatory `term: {tenant_id}` filter sourced server-side from `request.state.tenant_id`. The API never accepts a tenant filter from the client. Cross-tenant reads are structurally impossible.
- 3. **Shard affinity** — `routing=tenant_id` co-locates each tenant's documents on one primary shard, so queries hit one shard instead of fanning out across the cluster.
- 4. **Quota** — per-tenant token bucket in Redis via atomic Lua. State is `(tokens, updated_at)`; on each request, `refill = (now - updated_at) × refill_rate`, capped at `capacity`. `capacity = rps × burst_multiplier` (default 100), `refill_rate = rps`. Server-side `TIME` removes client clock skew. Default 50 rps per tenant, overrideable in the `tenants` row.
- 5. **Cache namespace** — keys prefixed with `sc:{tenant_id}:`. Cross-tenant cache poisoning is structurally impossible.

For tenants requiring physical isolation (compliance), the single shared index swaps for an alias resolver pointing at index-per-tenant. The code change is one resolver function.

Bounded timeouts

No code path can hang on a wedged backend. OpenSearch client timeout is 0.7s on the API and 5s on the indexer. Postgres `command_timeout` is 0.5s on the API and 2s on the indexer. Redis `socket_timeout` is 0.1s on the API. The search route catches OpenSearch exceptions and falls back to the stale cache; if no stale entry exists, it returns 503.

Trade-offs

Decision	Upside	Downside	Mitigation
Async indexing	Fast write path; indexer failures don't break writes	Search is eventually consistent	Read-your-write via <code>GET /documents/{id}</code>
Single shared OS index	Cheap, one index to operate	Noisy-neighbor risk between tenants	Per-tenant shard routing; escape hatch to index-per-tenant
Redis Streams over Kafka	No extra broker; at-least-once with consumer groups	Lower throughput ceiling, no infinite retention	Migrate at sustained ~50k msg/s
Header-based tenancy	Trivial to implement for the prototype	Spoofable by any client	Replace with signed JWT in production
Token-bucket rate limit	Smooth refill, returns remaining tokens cheaply	Hot key per tenant on a single Redis	Shard Redis by tenant when a single instance saturates